

บทที่ 2

ความรู้เบื้องต้นเกี่ยวกับ 3D Game Engine

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานโดยทั่วไป และที่เกี่ยวข้องในการพัฒนา 3D Game Engine จำทำให้ผู้อ่านทราบถึงประโยชน์ และสามารถเข้าใจในเนื้อหาในส่วนของบทที่ 4 ได้อย่างดียิ่งขึ้น ซึ่งทฤษฎีต่างๆในบทนี้จะไม่ได้กล่าวถึงอย่างละเอียดมากนัก แต่มากพอที่จะช่วยให้บรรลुวัตถุประสงค์ข้างต้นได้

2.1 ส่วนประกอบหลักของ 3D Game Engine

Game Engine เป็นเครื่องมือที่สามารถช่วยให้ผู้พัฒนาเกม สามารถมุ่งเน้นไปที่การพัฒนาเกมได้มากกว่า การพัฒนาส่วนประกอบอื่นๆให้สามารถทำงานได้ ส่วนประกอบเหล่านั้นได้แก่ส่วนของ Graphics ซึ่งจะเกี่ยวข้องโดยตรงกับการใช้งาน Graphics API ต่างๆ ส่วนของ Physics ซึ่งทำการจำลองการเคลื่อนที่และการชนกันของวัตถุโดยใช้วิธีต่างๆ ส่วนของ Sound ซึ่งใช้ในการเล่นเสียงประกอบในเกม

ใน 3D Game Engine นั้นควรประกอบไปด้วยส่วนที่จำเป็นดังต่อไปนี้

2.1.1 Mathematical Module

การพัฒนา 3D Game Engine ที่ไม่ขึ้นตรงกับ API ใดนั้น จำเป็นต้องมีส่วนของ Mathematical Module เป็นของตนเอง เนื่องจากไม่สามารถที่จะเรียกใช้เครื่องมือช่วยเหลือจาก API ใดได้อย่างเฉพาะเจาะจง ซึ่งในส่วนนี้จะต้องสามารถช่วยให้การคำนวณทางคณิตศาสตร์ที่เกี่ยวข้อง เป็นไปได้อย่างถูกต้อง สะดวก แม่นยำ และรวดเร็ว เนื่องจากส่วนนี้จะถูกเรียกใช้งานมากและบ่อยที่สุดเมื่อเทียบกับส่วนประกอบอื่น ความเร็วที่ใช้ในการทำงานจึงเป็นสิ่งสำคัญ โดยทั่วไปแล้วการทำ Profile เพื่อช่วยในการ Optimization จะแสดงให้เห็นว่าส่วนนี้จะส่งผลกับประสิทธิภาพโดยรวมของระบบมากที่สุด ส่วนนี้จึงต้องใช้ความระมัดระวังในการออกแบบและเขียนโปรแกรมอย่างมาก เพื่อให้เป็นไปตามจุดมุ่งหมายที่วางไว้ หัวข้อการคำนวณทางคณิตศาสตร์ที่สำคัญได้แก่ Vector, Matrix และ Quaternion เป็นต้น

2.1.2 Graphics Module

Module นี้ถือว่าเป็นส่วนที่สำคัญที่สุดส่วนหนึ่งในส่วนประกอบทั้งหมด เนื่องจาก เป็นส่วนแสดงผลภาพ นอกจากจะมีหน้าที่ติดต่อกับ Graphics API โดยตรงแล้ว ยังสามารถมีส่วนขยายเพื่ออำนวยความสะดวกให้กับผู้ใช้ หรือเพื่อให้มีลูกเล่นที่สวยงามน่าชมยิ่งขึ้น โดยที่ผู้ใช้ไม่จำเป็นต้องมีความรู้ทางทฤษฎีเกี่ยวกับลูกเล่นเหล่านั้นทั้งหมด เพียงแค่เข้าใจในหลักการง่ายๆ ก็สามารถ

ใช้งานได้ นอกจากนี้อาจจะมีส่วนของ Scene Management ที่ช่วยให้การแสดงผลเป็นไปอย่างมีประสิทธิภาพมากขึ้น

2.1.3 Physically Base Simulation Module

สามารถทำการจำลองการเคลื่อนไหวกของวัตถุชนิดแข็งไม่ยืดหยุ่นให้ใกล้เคียงความเป็นจริงมากที่สุดโดยอาศัยหลักทฤษฎีทางฟิสิกส์เกี่ยวกับจลพลศาสตร์มาช่วยในการจำลอง โดยประกอบไปด้วยส่วนของการตรวจจับการชนกันของวัตถุ (Collision detection) การตรวจหาปฏิสัมพันธ์อย่างง่ายหลังการชนของวัตถุแข็ง (Collision response of Rigid body) และ การจำลองการเคลื่อนที่ของวัตถุแข็ง (Rigid body simulation) โดยจะกล่าวถึงรายละเอียดและวิธีการในส่วนต่อไป

2.1.4 Sound Module

ส่วนนี้ใช้สำหรับ เล่นไฟล์เสียงที่ต้องการ และอาจใช้ควบคุมระดับและทิศทางของเสียงที่เล่นได้ โดยส่วนมาก API ที่ติดต่อจะมีความสามารถค่อนข้างสูง และสามารถใช้งานได้ง่าย ส่วนนี้จึงไม่มีความสลับซับซ้อนใดๆ เพียงแค่สามารถเรียกใช้ API ที่ต้องการได้เท่านั้น โดยส่วนนี้อาจจะเพิ่มเติมขึ้นมาคือส่วนของการควบคุมการใช้ทรัพยากรเท่านั้น

2.2 ลักษณะของ Graphics Module ใน Game Engine ที่มีอยู่ในปัจจุบัน

ในปัจจุบันส่วนของ Graphics Module ใน Game Engine ขั้นนำได้ถูกพัฒนาไปอย่างรวดเร็ว ทั้งนี้เนื่องจากความสวยงามของภาพสามารถนำไปใช้เป็นจุดเด่นทางการค้าได้ดี ในหัวข้อนี้จะแสดงถึงคุณสมบัติเด่นๆ ที่จำเป็น

ตารางต่อไปนี้จะแสดงคุณสมบัติทางด้านการแสดงผลและส่วนที่เกี่ยวข้องของ Source Engine ของ บริษัท Valve ที่ใช้ในเกม Half-life 2 เทียบกับความสามารถของ PC Engine (ชื่อโครงการนี้) ซึ่งมีเฉพาะความสามารถที่สำคัญพื้นฐานของ Game Engine ในปัจจุบัน

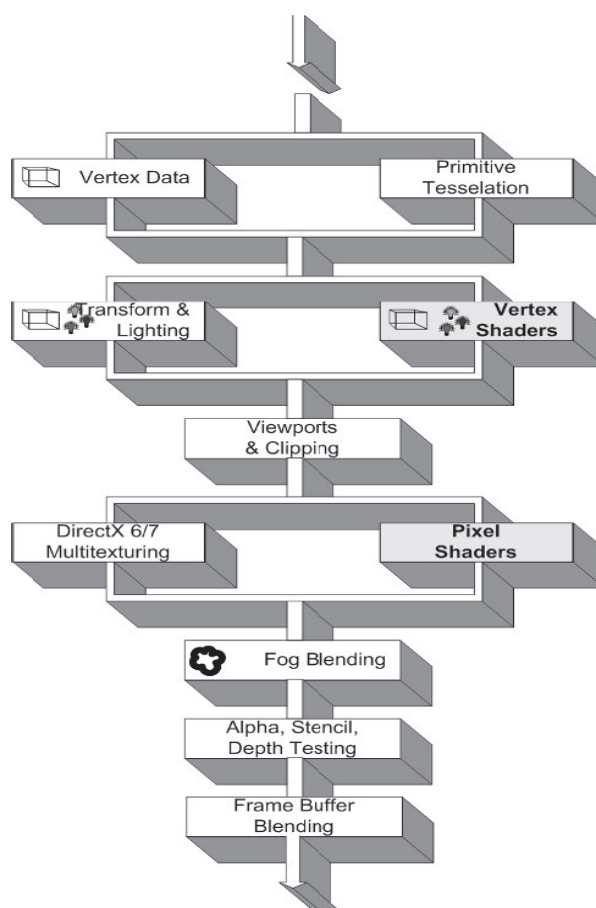
Feature	Source Engine	PC Engine
Lighting	Per-vertex, Per-pixel, Light mapping, Radiosity, Gloss maps:	Per-vertex, Light mapping
Shadow	Shadow Mapping:	-
Texturing	Basic, Multi-texturing, Bumpmapping, Mipmapping:	Basic, Multi-texturing, Bumpmapping
Shaders	Vertex, Pixel, High Level:	Vertex, Pixel

Scene Management	BSP, Portals, Occlusion Culling, LOD:	Scene graph (Basic Frustum Culling)
Animation	Skeletal Animation, Morphing, Facial Animation, Animation Blending:	Skeletal Animation
Meshes	Mesh Loading, Skinning, Progressive, Deformation:	Mesh Loading, Skinning
Special Effects	Environment Mapping, Billboarding, Particle System:	Billboarding
Terrain	Rendering, CLOD:	Rendering, CLOD

ตารางที่ 2.1 เปรียบเทียบความสามารถระหว่าง Source Engine กับ PC Engine [ข้อมูลอ้างอิงจาก 3D Game Engine Database – <http://www.devmaster.net/engines>]

2.3 แนะนำกระบวนการแสดงภาพ 3 มิติ

ในปัจจุบันเทคโนโลยีใหม่ๆของการ์ดแสดงผลอย่าง Vertex/Pixel Shader ได้เปิดโอกาสให้เรากำหนดกระบวนการการแสดงผลภาพสามมิติได้ด้วยตนเอง (Programmable Pipeline) ในบางขั้นตอนของกระบวนการ ดังนั้นผู้ใช้ Engine ควรจะเข้าใจถึง 3D Pipeline ในปัจจุบัน และมีความรู้พื้นฐานเกี่ยวกับเรื่อง Computer Graphics บ้าง ในส่วนนี้จึงขออนุญาตนำเสนอความรู้เบื้องต้นเหล่านี้



รูปที่ 2.1 3D Graphic Pipeline (With Vertex/Pixel Shader)

จากรูปที่ 2.1 จะเห็นได้ว่าในช่วงของกระบวนการ Transform & Lighting และ Texturing นั้น จะเปิดโอกาสให้ผู้ใช้งานสามารถทำการ Program เองได้ด้วย Vertex/Pixel Shader ดังนั้นจึงต้องเข้าใจเกี่ยวกับส่วนประกอบของ 3D Pipeline เพื่อที่จะสามารถใช้งานร่วมกันกับขั้นตอนอื่นๆได้อย่างถูกต้อง ส่วนเรื่องเกี่ยวกับ Vertex/Pixel Shader จะกล่าวถึงในส่วนถัดไป

2.3.1 Vertex Data

ข้อมูล Vertex จัดเป็นข้อมูลที่เป็น Input ให้กับ Graphic Pipeline องค์ประกอบของข้อมูล อาจจะแตกต่างกันออกไปตามลักษณะการใช้งาน อย่างไรก็ตามองค์ประกอบของข้อมูล Vertex ที่ขาดไม่ได้คือ Vertex Position หรือตำแหน่งของ Vertex นั้นเอง โดยข้อมูล Position นั้นสามารถเก็บอยู่ในรูปแบบของ Vector3 (x, y, z)

องค์ประกอบของ Vertex Data อื่นๆ ที่จำเป็นได้แก่ Vertex Color(r, g, b, a), Vertex Normal (x, y, z) และ Texture Coordinate (u, v) ซึ่งในส่วนของ Vertex Color นั้นจะใช้ก็ต่อเมื่อต้องการกำหนดสีให้กับ Vertex โดยตรงเท่านั้นและจะต้องไม่มีแสงเข้ามาเกี่ยวข้อง หากมีแสงเข้ามาเกี่ยวข้องจะต้องใช้ Vertex Normal ซึ่งแสดงถึงทิศทางของ Vertex เพื่อที่จะนำไปคำนวณผลกระทบที่เกิดจากแสง

ตัวอย่าง Structure ของ Vertex ที่ใช้โดยทั่วไปสามารถกำหนดได้ดังต่อไปนี้

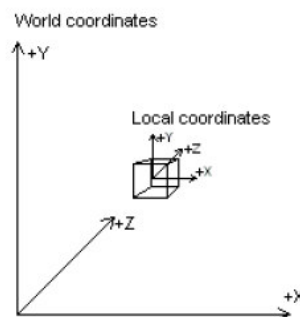
```

struct Vertex
{
    Vector3 Position;      //Position
    Vector3 Normal;        //Normal
    Vector2 UV;            //Texture Coordinate
};

```

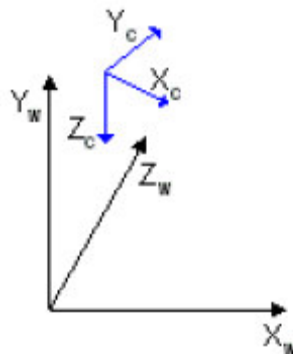
2.3.2 Transformation

ในขั้นแรกของ Pipeline ข้อมูล Vertex (Vertex Position) ส่วนมากจะอยู่ใน Local Coordinate System (Model Space) ซึ่งของแต่ละวัตถุจะแยกออกจากกัน Transformation ครั้งแรกของ Pipeline จะเป็นการแปลงจาก Model Space ไปเป็น World Space ซึ่งจะเป็น Unify Space ของทุกวัตถุ เรียกว่า World Transform



รูปที่ 2.2 World และ Local Coordinate (World Transform สามารถเป็นได้ทั้ง Translation Rotation และ Scaling)

ในลำดับต่อมาจะกล่าวถึงการแปลงจาก World Space ไปสู่ Camera Space เรียกว่า View Transform ซึ่งจะแสดงถึงการจัดวางตำแหน่งและทิศทางของกล้องที่ใช้แสดงผลนั่นเอง โดย Matrix Formula แสดงถึง View Transformation ($V = T \cdot R_z \cdot R_y \cdot R_x$)



รูปที่ 2.3 View และ World Coordinate

```
//ตัวอย่างการสร้าง View Transform Matrix
```

```
zaxis = normal(Eye - At)
```

```
xaxis = normal(cross(Up, zaxis))
```

```
yaxis = cross(zaxis, xaxis)
```

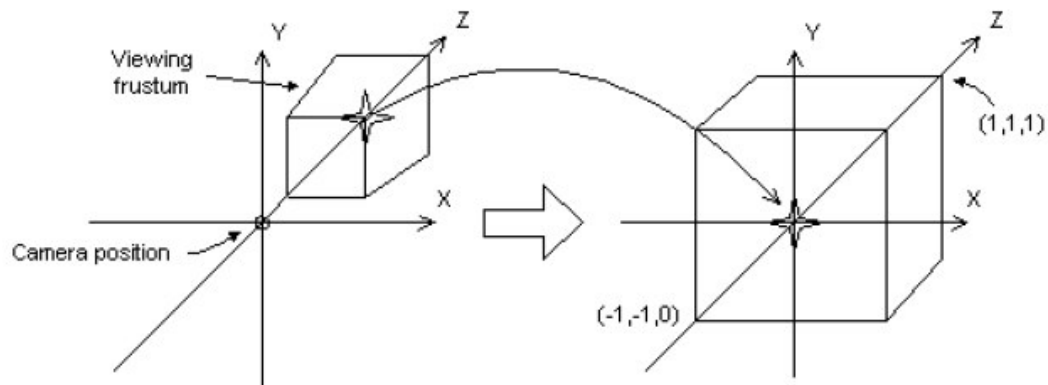
```
xaxis.x    yaxis.x    zaxis.x    0
```

```
xaxis.y    yaxis.y    zaxis.y    0
```

```
xaxis.z    yaxis.z    zaxis.z    0
```

```
-dot(xaxis, eye) -dot(yaxis, eye) -dot(zaxis, eye) 1
```

ต่อมาคือ Projection Transform ซึ่งในส่วนนี้วัตถุต่างๆจะถูก Scale ตามระยะห่างของวัตถุกับ Viewer เพื่อที่จะสามารถแสดงถึงความลึกของภาพได้ วัตถุที่อยู่ใกล้กว่าจะมีขนาดใหญ่กว่า และเช่นกันวัตถุที่อยู่ไกลกว่าจะมีขนาดเล็กกว่า เรียกว่า Homogeneous Space (Projection Space) แต่ก็มีบางชนิดของ Projection Transform ที่ไม่ทำการ Scale เช่น Affine หรือ Orthogonal Projection



รูปที่ 2.4 Projection Transform

```
//ตัวอย่างการสร้าง Projection Matrix จาก Field of View (FOV)
```

```
xScale  0    0    0
```

```
0    yScale  0    0
```

```
0    0    zf/(zf-zn)  1
```

```
0    0    -zn*zf/(zf-zn)  0
```

where:

```
yScale = cot(fovY/2)
```

```
xScale = aspect ratio * yScale
```

ขั้นสุดท้ายคือการทำ Clipping เพื่อที่จะตัด Vertex ที่มองไม่เห็นออก เพื่อที่ Rasterizer จะไม่ต้องเสียเวลาคำนวณสีและแสงเงา ถัดจาก Clipping ก็คือ Transformation ครั้งสุดท้ายที่ Vertices จะถูก Scale ตาม Viewport ที่กำหนดและเปลี่ยนเป็น Screen Coordinate เพื่อทำ Rasterization ให้ปรากฏเป็นภาพบนจอ

ตามปกติแล้ว Transform Matrix แบบต่างๆจะถูกนำมารวมเข้าด้วยกันเป็น World-View-Projection Matrix (WVP) เพื่อทำการ Apply ให้กับแต่ละ Vertex อย่างไรก็ตามผู้ใช้นี้ยังมีความจำเป็นต้องรู้ถึงพื้นฐานของ Transformation Matrix ที่อยู่ในรูปแบบของ Matrix 4x4 ซึ่งใช้ในการ Translation Rotation และ Scaling ด้วยสามารถสรุปได้ดังนี้

//Translation

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ T_x & T_y & T_z & 1 \end{bmatrix}$$

//Scaling

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

//Rotation about x-Axis

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

//Rotation about y-Axis

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

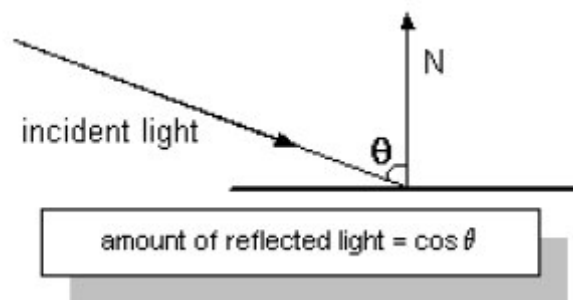
//Rotation about z-Axis

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.3.3 Material & Lighting

ก่อนที่จะสามารถพูดถึงเรื่องแสงได้นั้น ต้องพูดถึงเรื่องของ Material ก่อน เนื่องจาก Material แสดงถึงการที่ Polygon จะสะท้อน (Reflect) หรือเปล่งแสง (Emit) ออกมาในภาพ คุณสมบัติของ Material ได้แก่ Diffuse Reflection, Ambient Reflection, Light Emission และ Specular Highlight ซึ่งแต่ละคุณสมบัติสามารถแสดงได้ด้วย ColorARGB ซึ่งประกอบไปด้วยสีและค่า Alpha ซึ่งจะนำไปใช้ในกระบวนการ Alpha Blending ด้วย

Diffuse และ Ambient Reflection จะแสดงถึงคุณสมบัติในการสะท้อน Ambient และ Diffuse Light ในฉากของวัตถุ เนื่องจากในฉากจะประกอบไปด้วย Diffuse Light มากกว่า Ambient Light การสะท้อน Diffuse Light จึงเป็นตัวกำหนดสีของวัตถุ และเนื่องจากว่า Diffuse Light มีทิศทาง มุมระหว่างแสงและ Normal Vector จึงมีผลกับความเข้มของการสะท้อน โดยการสะท้อนจะมากที่สุดเมื่อทิศทางของแสงขนานกับทิศทางของ Normal Vector เมื่อมุมมากขึ้นก็จะสะท้อนน้อยลงดังรูปที่ 2.5



รูปที่ 2.5 แสดงการคำนวณความเข้มของการสะท้อน

ส่วน Ambient Reflection จะเหมือนกับ Ambient Light คือไม่มีทิศทางและมีผลต่อวัตถุทั้งชิ้นเท่าเทียมกันซึ่งจะเห็นได้ชัดเมื่อวัตถุไม่มี Diffuse light มากกระทำ

Emission คือการที่วัตถุสามารถเปล่งแสงออกมาได้ โดยเราสามารถให้ Emissive Property ทำให้ดูเหมือนวัตถุเป็นแหล่งกำเนิดแสงได้ ส่วน Specular Reflection สามารถทำให้วัตถุเป็นมันเงา โดยทั่วไป Material Structure จะสามารถกำหนดสี และค่าความเงาได้

องค์ประกอบแสง หรือ Light ประกอบไปด้วย 4 ส่วนเช่นเดียวกับ Material คือ Ambient Diffuse Emissive และ Specular โดยทั้งสี่ส่วนจะมีวิธีการคำนวณแตกต่างกันออกไป แต่ผลลัพธ์จะถูกนำมารวมกันดังต่อไปนี้

```
//การรวมแต่ละองค์ประกอบของแสงเป็น Global Illumination
```

```
Global Illumination = Ambient Light + Diffuse Light + Specular Light + Emissive Light
```

```
//Ambient Light
```

```
Ambient Lighting = MaterialAmbientColor*[Global AmbientColor + LightAmbientColor]
```



```
//Diffuse light
Diffuse Lighting = DiffuseColor*LightDiffuseColor*(Normal.Direction)

//Specular Light
Specular Lighting = SpecularColor * [LightSpecularColor * (Normal • HalfWay)^Power *]

//Emissive Light
Emissive Lighting = EmissiveColor

//Attenuation
Attenuation = 1/( ConstantAttenuate + LinearAttenuate * Distance + QuadraticAttenuate * d^2)
```

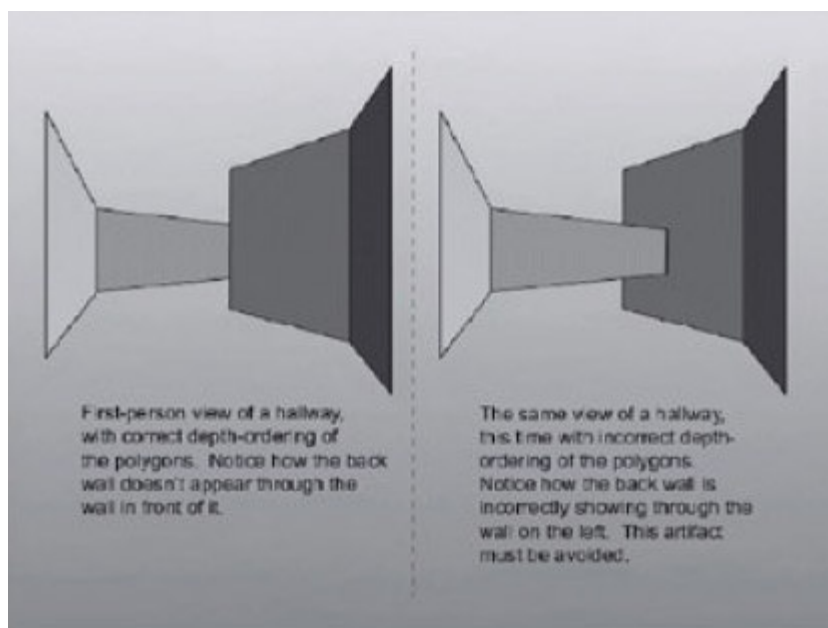
แสงยังสามารถแบ่งได้หลายประเภทโดยประเภทที่นิยมได้แก่ Point และ Directional Light ซึ่งมีคุณสมบัติคล้ายคลึงกัน Point Light จะเทียบได้กับหลอดไฟ คือมีจุดกำเนิด และ เปล่งแสงไปรอบๆ แต่ Directional จะเทียบได้กับดวงอาทิตย์ส่องแสงมายังโลก คือมีแต่ทิศทางของแสง แต่ไม่คิดตำแหน่งของแสงเนื่องจากถือว่าอยู่ห่างจากวัตถุมาก

2.3.4 Texture

Texture mapping คือการนำภาพ 2มิติ มา map เป็นลวดลายให้กับวัตถุ ข้อมูลที่สำคัญ ได้แก่ Texture Image หรือรูปที่จะใช้ และ Texture Coordinate ของแต่ละ Vertex โดย Texture Coordinate แสดงถึงบริเวณของรูปที่จะใช้ในการ Map ให้กับ Vertex นั้นโดยในกระบวนการ Rasterization จะมีการทำ Sampling Texture Image เพื่อนำค่าสีของรูปมาใส่ เนื่องจาก Texture mapping มีลูกเล่นมากมาย อีกทั้งยังสามารถใช้หลาย Texture พร้อมกันได้จึงขออธิบายในส่วนนี้ไว้เพียงเท่านี้ และจะอธิบายเพิ่มเติมควบคู่ไปกับการออกแบบ Texture Class

2.3.5 Buffers

Buffer ที่ใช้โดยทั่วไปสามารถแบ่งได้สามประเภทคือ Front/Back Buffer หรือ Frame Buffer, Depth Buffers และ Stencil Buffer ซึ่งแต่ละชนิดมีหน้าที่ต่าง ๆ กัน โดย Frame Buffer จะเก็บค่าสีและค่า Alpha ส่วน Depth Buffer จะใช้ในการแก้ปัญหา Depth Problem ดังในรูปที่ 2.6 โดยจะเก็บค่าความลึกไว้ดังในรูปที่ 2.7 ส่วน Stencil Buffer เป็น Buffer อเนกประสงค์สามารถใช้ในการเลือกส่วนที่ต้องการแสดงผลด้วยวิธีต่างๆได้ เปรียบได้กับการทาสีโดยปิดส่วนที่ไม่ต้องการเอาไว้ก่อน

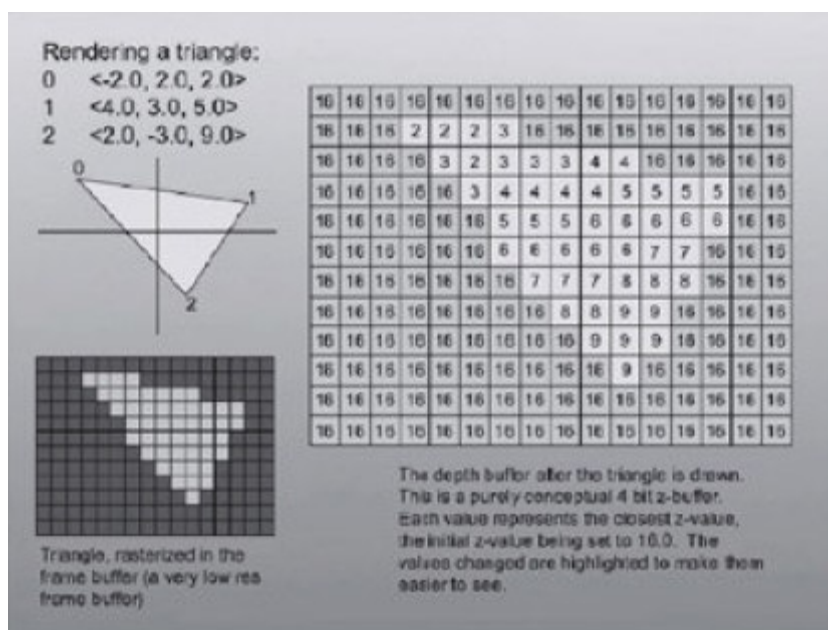


A

B

รูปที่ 2.6 แบบ A แสดงผลลัพธ์ที่ต้องการ และ แบบ B แสดงการแก้ปัญหาโดยการทำ Polygon

Depth Ordering



รูปที่ 2.7 การแก้ปัญหาโดยใช้ Depth Buffer

2.3.6 Graphics API

ในการเขียนโปรแกรมติดต่อ Graphic Hardware (Graphics Card) ให้แสดงผลภาพสามมิติ นั้นจำเป็นต้องอาศัยตัวกลางอย่าง Graphics API เข้ามาช่วยทำให้ง่ายขึ้น ในปัจจุบันมีทางเลือก ใหญ่ๆเพียง 2 ทางเลือกคือ Direct Graphics (Microsoft) และ OpenGL (Silicon Graphics) อย่างไรก็ตามผู้พัฒนาก็สามารถเลือกที่จะเขียน Driver และ Software 3D Graphics and

Rasterization เองได้ Game Engine ที่ดีควรที่จะออกแบบให้สามารถเลือก API ในการทำงานได้อย่างอิสระ โดยให้ส่วนหลักของ Engine ไม่ขึ้นตรงกับ Graphics API โดยเฉพาะเจาะจง

2.4. ลักษณะของ Physically Based Simulation Module ใน Game Engine ที่มีอยู่ในปัจจุบัน

ในปัจจุบัน Physics Engine ได้มีการพัฒนาอย่างมากซึ่งได้มีความสามารถนอกเหนือจากการ ตรวจสอบการชน ,การเคลื่อนที่ของวัตถุแข็ง , ฟิสิกส์พื้นฐาน คือสามารถที่จะทำการตอบสนองหลังจากเกิดการชนกัน ได้กับวัตถุเกือบทุกชนิด สามารถทำการเปลี่ยนแปลง ภาพและเสียงจาก การคำนวณทางฟิสิกส์ มีความสามารถหาความสัมพันธ์ของพลังงานในการเคลื่อนที่ อนิเมชันของโบน ฟิสิกส์เกี่ยวกับยานพาหนะ แรงลอยตัวของวัตถุในของเหลว

2.5. แนะนำกระบวนการ Physically Based Simulation

ในการจำลองนั้นจะมีสามส่วนด้วยกันคือ

1. การเคลื่อนที่เชิงเส้น
2. การเคลื่อนที่เชิงมุม
3. การตรวจสอบการชน

2.5.1. การเคลื่อนที่เชิงเส้น

เริ่มจาก วัตถุที่เคลื่อนที่ได้ต้องมีมวล ของวัตถุและถ้าไม่มีความเร็วเริ่มต้นวัตถุจะไม่เคลื่อนที่แต่ถ้ามีแรงไปกระทำกับวัตถุ ทำให้วัตถุเกิดการเคลื่อนที่ตาม หลัก ทฤษฎีทางฟิสิกส์ โดยการจำลองจะนำแรงที่กระทำกับวัตถุทั้งหมด มารวมกัน เป็นแรงลัพธ์ แล้วนำแรงลัพธ์ที่ได้ไปคำนวณหาความเร็วที่กระทำวัตถุ เมื่อได้ความเร็วที่กระทำกับวัตถุ ณ. เวลานั้นแล้วก็จะเอาไปรวมกับความเร็วเดิมของวัตถุทำให้ได้ความเร็วใหม่ และเมื่อได้ความเร็วใหม่ก็จะนำความเร็วตัวนี้ไปบวกกับตำแหน่ง เดิมของวัตถุ ได้ตำแหน่งใหม่ของวัตถุในระบบสามมิตินั่นเอง โดยทุกขั้นตอนจะเป็นการกระทำ ณ เวลาหนึ่งเท่านั้น

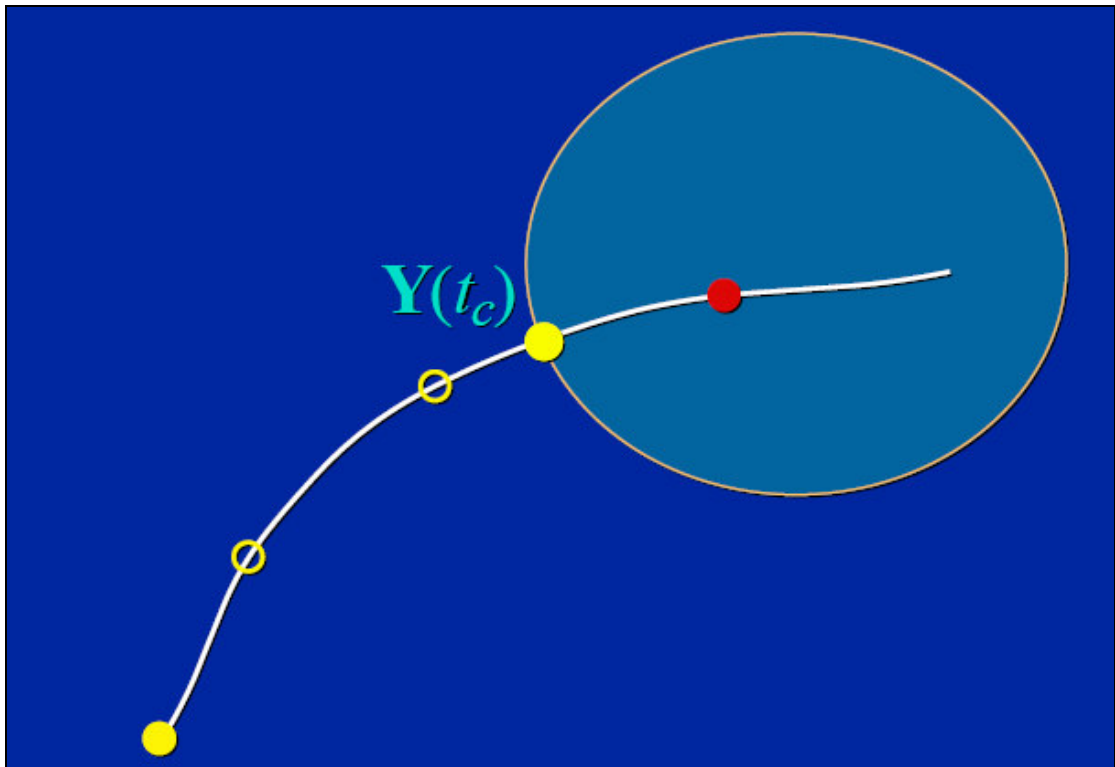
2.5.2. การเคลื่อนที่เชิงมุม

ในการเคลื่อนที่เชิงมุมก็เช่นเดียวกับการเคลื่อนที่เชิงเส้น โดยเราจะนำแรงที่กระทำมารวมเป็นแรงลัพธ์ แล้วนำมาหาว่าแรงลัพธ์นั้นกระทำผ่านจุดศูนย์กลางวัตถุหรือไม่เพราะถ้ากระทำผ่านจุดศูนย์กลางของวัตถุ จะทำให้แรงที่กระทำนั้นไม่ทำให้เกิดความเร็วเชิงมุม แต่ถ้าไม่ผ่านจุดศูนย์กลางเราก็จะนำแรงลัพธ์นั้นมาหาความเร็วเชิงมุมตาม หลักทฤษฎีทางฟิสิกส์ แล้วนำความเร็วที่ได้ไปทำการบวกเพิ่มกับความเร็วเชิงมุมเดิม จากนั้นก็หมุนวัตถุ

ด้วยความเร็วเชิงมุมใหม่ที่ได้ทำให้วัตถุที่เคลื่อนที่นั้นหมุนไปด้วย โดยทุกขั้นตอนจะเป็นการกระทำ ณ เวลาหนึ่งเท่านั้น

2.5.3. การตรวจสอบการชน

เมื่อวัตถุเคลื่อนที่ได้ ก็จะมีโอกาสที่จะเคลื่อนที่มาชนกันดังนั้นจึงต้องมีการตรวจสอบการชนกันของวัตถุในการเคลื่อนที่แต่ละ ครั้งด้วย เพราะถ้าไม่มีการตรวจสอบการชนกัน จะเกิดการทับกันของวัตถุที่ชนกันเหมือนกับว่าไม่มีวัตถุอีกชิ้นอยู่ทำให้ไม่สมจริง ซึ่งการตรวจสอบการชนสามารถดูได้จากรูปที่ 2.8



รูปที่ 2.8 แสดงการจำลองการตรวจสอบการชนกันของวัตถุในแต่ละการเคลื่อนที่

จากรูปที่ 2.8 จะเห็นได้ ว่าในการเคลื่อนที่แต่ละครั้งนั้นจะมีการตรวจสอบว่าวัตถุนั้นได้เคลื่อนที่เข้าไปใน วงกลมรัย ซึ่งถ้ามีการเคลื่อนที่เข้าไปแล้วก็จะทำการยับจุดนั้นออกมาเป็นเวลา เท่ากับ Contact time เวลาที่วัตถุเริ่มชน เพื่อให้จุดที่ได้นั้นอยู่ที่พื้นผิวของวัตถุ ซึ่งหลังจากที่ตรวจสอบได้ว่าวัตถุสองชิ้นชนกันแล้วก็ต้องมาคำนวณว่าจะเกิดอะไรหลังจากการชนกัน โดยในส่วนนี้สามารถกำหนดได้หลายรูปแบบ เช่น ทิศทางของวัตถุที่เคลื่อนที่ชนกันเกิดการเปลี่ยนแปลงหลังจากที่ชนรวมไปถึงความเร็วของวัตถุ ก่อนชนกับหลังชนด้วย หรือ อาจจะมีการเปลี่ยนสีของวัตถุหลังจากที่ชนกันซึ่งในส่วนนี้ขึ้นกับผู้ออกแบบว่าต้องการให้เป็นไปในทิศทางใด